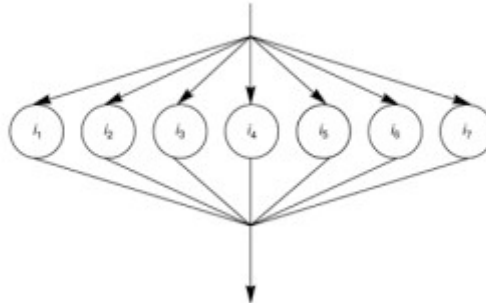


Sumario

1. Introducción.....	2
2. Lenguaje C.....	4
2.1 Compilación y ejecución.....	7
2.2 Ejemplos.....	8
2.3 Actividades.....	11
3. Llamadas al sistema.....	12
4. Fork y exec.....	15
4.1 Ejemplo.....	17
4.2 Ejemplo.....	19
4.3 Actividades.....	21
5. Tubería.....	23
5.1 Ejemplo.....	24
5.2 Ejemplo 2.....	26
5.3 Actividades.....	29
6. Interprete de ordenes.....	31
6.1 Actividades.....	33
7 Ejecución en segundo plano.....	34
7.1 Actividades.....	36

1. Introducción



La programación multiproceso es una técnica de programación que se utiliza para aprovechar los múltiples núcleos o procesadores de una computadora o sistema para realizar tareas concurrentes o paralelas. En lugar de ejecutar un programa en un solo hilo de ejecución, la programación multiproceso permite que múltiples hilos de ejecución (procesos) se ejecuten de manera simultánea para realizar diferentes tareas.

Algunos conceptos clave relacionados con la programación multiproceso:

1. **Proceso:** Un proceso es un programa en ejecución que tiene su propio espacio de memoria y recursos, como registros de CPU y pilas. Los procesos pueden ejecutarse de manera independiente y aislada.
2. **Hilo de Ejecución (Thread):** Un hilo de ejecución es una unidad más pequeña dentro de un proceso que puede ejecutarse de manera independiente. Los hilos comparten el espacio de memoria y recursos de su proceso principal. Los hilos permiten una programación más concurrente y paralela dentro de un proceso.

3. **Concurrencia:** La concurrencia se refiere a la capacidad de realizar múltiples tareas aparentemente al mismo tiempo. Los procesos y los hilos de ejecución pueden ejecutarse concurrentemente, lo que permite una mejor utilización de los recursos de hardware.
4. **Paralelismo:** El paralelismo es una forma específica de concurrencia en la que los procesos o hilos se ejecutan realmente al mismo tiempo, aprovechando los núcleos de CPU múltiples. El paralelismo puede mejorar significativamente el rendimiento de las aplicaciones en sistemas con múltiples núcleos.
5. **Comunicación y Sincronización:** En la programación multiproceso, los procesos o hilos pueden compartir datos y recursos. Sin embargo, es importante gestionar la comunicación y la sincronización adecuadamente para evitar problemas como condiciones de carrera y bloqueos. Se utilizan mecanismos como semáforos, mutexes y variables de condición para lograr la sincronización.
6. **Programación Concurrente vs. Paralela:** La programación concurrente se refiere a la ejecución de tareas en el mismo período de tiempo, mientras que la programación paralela se refiere a la ejecución simultánea de tareas en múltiples núcleos o procesadores.

2. Lenguaje C

El lenguaje de programación C es uno de los lenguajes de programación más antiguos y ampliamente utilizados en la industria de la programación. Fue creado en la década de 1970 por Dennis Ritchie en los laboratorios Bell de AT&T como una evolución del lenguaje de programación B. C ha sido influyente en el desarrollo de muchos otros lenguajes de programación y sistemas operativos, y su legado perdura hasta el día de hoy.

En el tema que nos encontramos, Programación multiproceso, los programas de creación y gestión de procesos concurrentes, serán implementados en lenguaje C. Para este menester se hubiesen podido utilizar otros lenguajes más vigentes como por ejemplo Java, Python y otros, no obstante se hará uso del lenguaje C por la facilidad y cercanía con el sistema operativo Linux en donde se ejecutarán los procesos concurrentes así creados.

Aquí tienes una introducción al lenguaje de programación C:

1. **Sintaxis Simple:** C es conocido por su sintaxis simple y elegante. Utiliza una combinación de palabras clave y operadores para realizar operaciones en datos.
2. **Compilado:** Los programas escritos en C se compilan en código máquina antes de ejecutarse. Esto significa que se traducen completamente antes de su ejecución, lo que puede hacer que los programas en C sean muy eficientes.
3. **Portabilidad:** C es conocido por su portabilidad, lo que significa que un programa escrito en C puede ejecutarse en una variedad de sistemas operativos y arquitecturas de hardware con pocos cambios.
4. **Estructuras de Control:** C admite estructuras de control como *if*, *else*, *while*, *for*, *switch*, que permiten la toma de decisiones y la repetición en programas.

5. **Punteros:** C es uno de los pocos lenguajes que permite el uso de punteros. Los punteros son variables que almacenan direcciones de memoria y son esenciales para la manipulación de datos y la gestión de memoria.
6. **Biblioteca Estándar:** C proporciona una biblioteca estándar que contiene una amplia gama de funciones que realizan tareas comunes, como entrada/salida, manejo de cadenas, matemáticas, etc.
7. **Estructuras y Uniones:** C permite definir estructuras de datos personalizadas utilizando `struct` y uniones con `union`, lo que facilita la creación de tipos de datos complejos.
8. **Modularidad:** Los programas en C se dividen generalmente en funciones, lo que facilita la modularidad y la reutilización del código.
9. **Dinámica y Estática:** C permite la asignación dinámica y estática de memoria, lo que brinda flexibilidad en la gestión de recursos.
10. **Alto Rendimiento:** C se utiliza comúnmente en el desarrollo de sistemas operativos, controladores de dispositivos y aplicaciones que requieren un alto rendimiento.
11. **Comunidad Activa:** A pesar de su edad, C sigue siendo un lenguaje activamente mantenido y utilizado en la industria, con una comunidad de programadores y una gran cantidad de recursos disponibles en línea.
12. **Herramientas de Desarrollo:** Existen muchas herramientas de desarrollo para C, incluidos compiladores como GCC, entornos de desarrollo integrados (IDE) como Code::Blocks y Visual Studio Code, y depuradores como GDB.

Programación multiproceso con el lenguaje C

C es un lenguaje poderoso y versátil que se utiliza en una amplia variedad de aplicaciones, desde sistemas embebidos hasta desarrollo de software de alto rendimiento.

A modo de ejemplo, seguidamente se puede ver un ejemplo de programa en lenguaje C que muestra en la salida standar el mensaje «¡ *Hola, Mundo!*»

```
#include <stdio.h>

int main() {

    printf("¡Hola, Mundo!\n");

    return 0;

}
```

La edición de programas en lenguaje C puede ser realizada en diferentes IDE'S como el Visual Studio, Eclipse, etc. Por las características y contenido del curso, en la edición de programas en lenguaje C se hará uso de editores de texto plano como por ejemplo el bloc de notas o el SciTE.

2.1 Compilación y ejecución

Si por ejemplo, se pretende compilar el programa del ejemplo anterior denominado *prg1.c* y crear, en el caso de ausencia de errores, el correspondiente programa ejecutable, habrá que escribir la orden siguiente:

```
gcc -o p1 prg1.c
```

Si todo ha ido bien y no ha habido ningún error sintáctico se habrá generado el fichero ejecutable *p1*.

Para ejecutar el mencionado archivo habrá que escribir:

```
./p1
```

2.2 Ejemplos

- Este programa solicita al usuario que ingrese dos números, los suma y muestra el resultado.:

```
#include <stdio.h>

int main() {
    int num1, num2, suma;
    printf("Ingrese el primer número: ");
    scanf("%d", &num1);
    printf("Ingrese el segundo número: ");
    scanf("%d", &num2);
    suma = num1 + num2;
    printf("La suma de %d y %d es %d\n", num1, num2, suma);
    return 0;
}
```

- Este programa convierte una temperatura en grados Celsius a grados Fahrenheit.

```
#include <stdio.h>

int main() {
    float celsius, fahrenheit;
    printf("Ingrese la temperatura en grados Celsius: ");
    scanf("%f", &celsius);
    fahrenheit = (celsius * 9/5) + 32;
    printf("%.2f grados Celsius son equivalentes a %.2f grados Fahrenheit\n", celsius, fahrenheit);
    return 0;}
}
```

- Programa en C que utiliza una declaración condicional `if-else` para determinar si un número introducido por el usuario es par o impar:

```
#include <stdio.h>

int main() {

    int numero;

    printf("Ingrese un número entero: ");

    scanf("%d", &numero);

    // Utilizamos el operador % para calcular el residuo de la división por 2.

    // Si el residuo es 0, el número es par; de lo contrario, es impar.

    if (numero % 2 == 0) {

        printf("%d es un número par.\n", numero);

    } else {

        printf("%d es un número impar.\n", numero);

    }

    return 0;

}
```

- Programa en C que utiliza un bucle `for` para mostrar en la consola la suma de los primeros números enteros hasta un número `n` introducido por el usuario:

```
#include <stdio.h>

int main() {

    int n;

    int suma = 0;

    printf("Ingrese un número entero positivo: ");

    scanf("%d", &n);

    if (n < 1) {

        printf("El número ingresado no es válido. Debe ser un entero positivo.\n");

        return 1; // Salida con error

    }

    for (int i = 1; i <= n; i++) {

        suma += i;

    }

    printf("La suma de los primeros %d números enteros es: %d\n", n, suma);

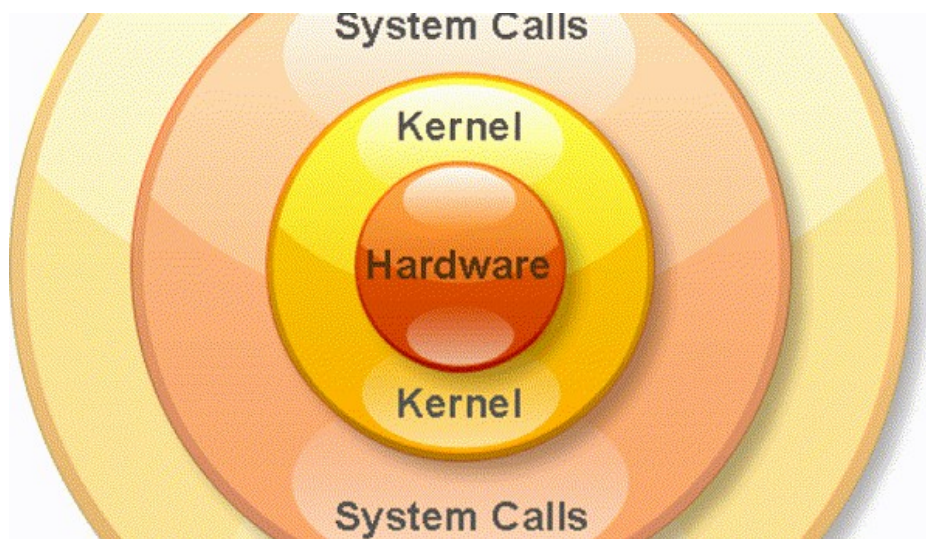
    return 0;}
```

2.3 Actividades

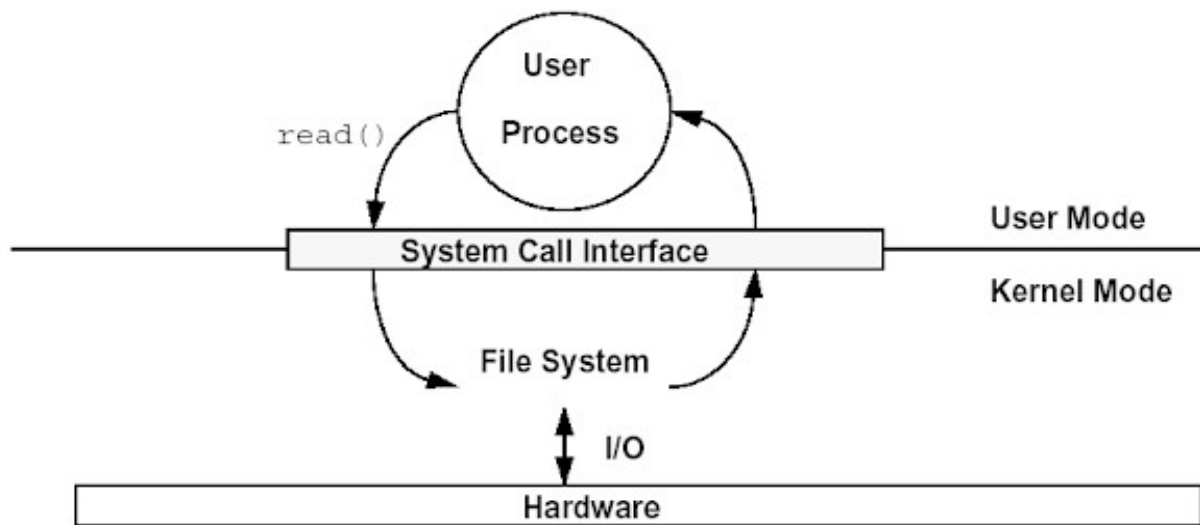
1. Desde la consola de Linux, mediante el comando gcc, compila y ejecuta los ejemplos de programas anteriores.
2. **Calculadora simple:** Crea un programa que permita al usuario realizar operaciones matemáticas básicas como suma, resta, multiplicación y división. El programa debe solicitar dos números y una operación al usuario, y luego mostrar el resultado.
3. **Verificador de números primos:** Desarrolla un programa que solicite al usuario ingresar un número y determine si es primo o no. Luego, muestra un mensaje que indique si el número es primo o no.
4. **Calculadora de factorial:** Escribe un programa que pida al usuario un número entero positivo y calcule su factorial utilizando un bucle `for`. Luego, muestra el resultado.
5. **Conversor de unidades:** Crea un programa que permita al usuario convertir entre diferentes unidades de medida, como metros a pies, kilogramos a libras, etc. El programa debe solicitar el valor a convertir, la unidad de origen y la unidad de destino, y mostrar el resultado de la conversión.
6. **Generador de tablas de multiplicar:** Desarrolla un programa que solicite al usuario un número y muestre la tabla de multiplicar correspondiente para ese número, desde 1 hasta 10.

3. Llamadas al sistema

En la ilustración siguiente los círculos concéntricos representan los diferentes niveles en un equipo informático. Cuando el usuario o aplicación solicitan algo que requiere el uso del hardware (acceso a una unidad de disco, gestión de procesos, impresión de un documento, etc.), se produce una **llamada al sistema** para que el código del núcleo del sistema operativo, a modo de intermediario entre el hardware y el usuario, tome temporalmente el control e interactúe con el hardware para solventar la petición efectuada por el usuario o aplicación. El círculo más externo de la gráfica siguiente representa al usuario o a la aplicación.



En la imagen que se puede ver a continuación, representa un ejemplo de llamada al sistema efectuada a partir de la solicitud de lectura de un fichero en disco (read) efectuada por una aplicación. Tras ejecutarse el `read()`, se activa una llamada al sistema para que el núcleo del sistema operativo tome riendas sobre el `assumpte`, indicando al disco que acceda al fichero solicitado para, de esta manera, devolver al programa el contenido del archivo que pretendía leer.



Una llamada al sistema (system call en inglés) en programación es una interfaz que permite a un programa de usuario solicitar servicios o recursos del sistema operativo. Estas llamadas al sistema actúan como puentes entre las aplicaciones de usuario y el kernel (núcleo) del sistema operativo, permitiendo a las aplicaciones realizar tareas que requieren acceso a funcionalidades del sistema, como el acceso al sistema de archivos, la gestión de procesos, la comunicación con dispositivos de hardware y otras operaciones esenciales.

Las llamadas al sistema proporcionan una capa de abstracción para que las aplicaciones interactúen con el hardware y el núcleo del sistema operativo sin tener que conocer los detalles de implementación específicos de cada sistema. Algunos ejemplos de llamadas al sistema comunes incluyen:

1. **open()**: Utilizada para abrir archivos en el sistema de archivos.
2. **read()** y **write()**: Para leer y escribir datos en archivos o dispositivos de entrada/salida.
3. **fork()**, **exec()**, y **wait()**: Utilizadas para la creación y gestión de procesos.
4. **malloc()** y **free()**: Para la asignación y liberación de memoria dinámica.
5. **socket()**, **bind()**, **connect()**, y **send()**: Para la comunicación de red en aplicaciones.
6. **exit()**: Para terminar la ejecución de un proceso.

El sistema operativo proporciona una serie de llamadas al sistema que las aplicaciones pueden invocar mediante funciones o instrucciones específicas según el lenguaje de programación que se utilice. Cuando una aplicación realiza una llamada al sistema, el kernel toma el control temporalmente, realiza la operación solicitada y devuelve el resultado a la aplicación.

Las llamadas al sistema son fundamentales para el funcionamiento de los programas y la interacción con el hardware y el sistema operativo en un sistema informático.

4. Fork y exec

fork y **exec** son dos **llamadas al sistema** fundamentales en la programación de sistemas Unix y Linux que permiten la creación y ejecución de procesos en C y otros lenguajes de programación. Tienen diferentes propósitos y funcionalidades:

1. **fork()**:

- **fork** se utiliza para crear un nuevo proceso en el sistema operativo.
- La llamada **fork** crea una copia exacta del proceso actual (proceso padre) en un nuevo proceso (proceso hijo).
- Ambos procesos (padre e hijo) continúan ejecutando el mismo código a partir del punto en el que se llamó a **fork**, pero tienen diferentes espacios de memoria y, por lo tanto, son procesos independientes.
- La función **fork** devuelve un valor diferente en cada proceso. En el proceso padre, devuelve el ID del proceso hijo, mientras que en el proceso hijo, devuelve 0.
- **fork** se utiliza para lograr la concurrencia en programas, permitiendo que múltiples tareas se ejecuten en paralelo. Los procesos pueden comunicarse entre sí a través de mecanismos como pipes o sockets.

2. `exec()`:

- `exec` se utiliza para reemplazar la imagen actual del proceso con una nueva imagen de programa.
- Cuando se llama a una de las funciones `exec`, el sistema operativo carga un nuevo programa en la memoria del proceso actual y comienza a ejecutar ese programa en lugar del programa original.
- `exec` es útil cuando deseas que un proceso realice una tarea diferente, como ejecutar un programa externo (por ejemplo, un comando del sistema), después de haber creado el proceso utilizando `fork`.
- Ejemplos de funciones `exec` incluyen `exec l`, `execv`, `exec le`, etc. Cada una de estas funciones permite pasar argumentos al nuevo programa y especificar el entorno del proceso.

En resumen, `fork` se utiliza para crear nuevos procesos a partir del proceso actual, mientras que `exec` se utiliza para reemplazar la imagen del programa actual con la de otro programa. Juntas, estas llamadas al sistema son fundamentales para la gestión de procesos y la ejecución de programas en sistemas Unix y Linux, lo que permite la creación de aplicaciones concurrentes y la ejecución de programas externos desde un programa en C

4.1 Ejemplo

En el siguiente código un proceso padre creará un proceso hijo para que este último ejecute la orden del sistema 'ls'. La creación del proceso hijo conlleva la ejecución de dos llamadas al sistema:

- **fork()**: El proceso padre crea un proceso hijo que es una copia exacta de sí mismo.
- **execl**: El código del proceso hijo que fue creado por el padre (a su imagen y semejanza) es sustituido por el código del comando linux *ls* (visualiza el contenido (ficheros y directorios) de una carpeta).

```
#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

int main() {

    pid_t pid;

    // Crear un nuevo proceso

    pid = fork();

    if (pid == -1) {

        // Error al crear el proceso hijo

        perror("Error al crear el proceso hijo");

        return 1;    }
```

```
else if (pid == 0) {  
    // Código ejecutado por el proceso hijo  
    printf("Proceso hijo ejecutando 'ls'\n");  
  
    // Reemplazar la imagen del proceso hijo con 'ls'  
    execl("/bin/ls", "ls", NULL);  
  
    // Si execl tiene éxito, el código a continuación no se ejecutará  
    perror("Error al ejecutar 'ls'");  
  
    return 1;  
} else {  
    // Código ejecutado por el proceso padre  
    printf("Proceso padre esperando al hijo...\n");  
  
    // Esperar a que el proceso hijo termine  
    int status;  
  
    wait(&status);  
  
    if (WIFEXITED(status)) {  
        printf("El proceso hijo ha terminado con estado %d\n", WEXITSTATUS(status));  
    } else {  
        printf("El proceso hijo ha terminado de manera anormal\n");  
    }  
}  
  
return 0;  
}
```

4.2 Ejemplo

En el siguiente ejemplo, nuevamente, un proceso padre crea un proceso hijo (*fork()*), como clon o copia exacta de sí mismo. A continuación, el proceso hijo se independiza del padre mediante la llamada al sistema *execlp("ps" "ps", "-e", NULL)*, ejecutando en ese momento su propio código para mostrar en consola la lista de procesos activos existentes en el sistema junto a su identificador o pid.

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

int main() {

    pid_t pid;

    // Crear un nuevo proceso hijo

    pid = fork();

    if (pid == -1) {

        perror("Error en fork");

        exit(1);    }
```

```
else if (pid == 0) {  
  
    // Código ejecutado por el proceso hijo  
  
    printf("Soy el proceso hijo (PID: %d)\n", getpid());  
  
    // Ejecutar el comando "ps" para mostrar los procesos activos  
  
    execlp("ps", "ps", "-e", NULL);  
  
    // Si execlp() tiene éxito, esta parte del código no se ejecuta  
  
    perror("Error en execlp");  
  
    exit(1);  
  
} else {  
  
    // Código ejecutado por el proceso padre  
  
    printf("Soy el proceso padre (PID: %d) y mi hijo es %d\n", getpid(), pid);  
  
    // Esperar a que el proceso hijo termine  
  
    wait(NULL);  
  
    printf("El proceso hijo ha terminado\n");  
  
}  
  
return 0;  
  
}
```

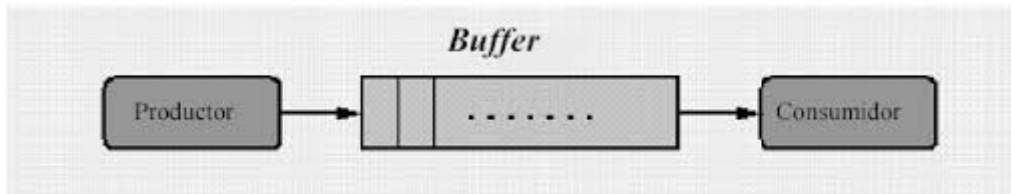
4.3 Actividades

1. Compila y ejecuta los dos programas de los ejemplos anteriores.
2. Programa que ejecuta el comando "date" para mostrar la fecha y hora actual en la consola.
3. Programa que crea un nuevo proceso hijo que imprime un mensaje en la consola, mientras que el proceso padre imprime otro mensaje diferente.
4. Programa que crea un nuevo proceso hijo que muestra en la consola la lista de archivos en un directorio específico (el directorio se especifica como argumento).
5. Programa que crea un proceso hijo que ejecuta un comando de usuario (especificado por el usuario) utilizando la llamada al sistema `execvp()`.
6. Programa que crea un proceso hijo que ejecuta un programa de usuario (especificado por el usuario) utilizando la llamada al sistema `execv()`.
7. Programa que crea dos procesos hijos mediante la llamada al sistema `fork`. Cada uno de ellos imprime un mensaje diferente en la consola.
8. Programa que crea un proceso hijo, mediante la llamada al sistema `fork`, que calcula e imprime la suma de los primeros N números enteros (N se especifica como argumento).

Programación multiproceso con el lenguaje C

9. Modifica el programa anterior para que además de realizar la llamada al sistema `fork`, realice la llamada al sistema `execv`.
10. Programa que crea un proceso hijo que verifica si un número ingresado por el usuario es primo o no.
11. Programa que crea un proceso hijo que calcula el factorial de un número ingresado por el usuario y muestra el resultado.
12. Programa que crea un proceso hijo que genera una tabla de multiplicar de un número ingresado por el usuario (desde 1 hasta 10) y muestra la tabla.
13. Cada uno de estos ejercicios te brinda la oportunidad de practicar el uso de `fork()` y `exec` o `execlp()` para la creación y ejecución de procesos en C. Puedes combinar estos ejercicios con el manejo de argumentos de línea de comandos y la entrada/salida estándar para crear programas más completos.

5. Tubería



En programación multiproceso, una tubería (pipe en inglés) es un mecanismo que permite la comunicación bidireccional entre dos procesos. Una tubería se utiliza para transmitir datos desde un proceso (el proceso padre) a otro proceso (el proceso hijo) o viceversa.

La utilidad principal de una tubería es permitir que los procesos compartan información de manera eficiente. Puede utilizarse para enviar datos desde un proceso a otro para su procesamiento o para sincronizar la ejecución de procesos, permitiendo que uno espere a que el otro complete una tarea antes de continuar.

En resumen, una tubería es una herramienta fundamental en la programación multiproceso que facilita la comunicación y la cooperación entre diferentes procesos en un sistema computacional.

5.1 Ejemplo

En el programa siguiente, un proceso padre creará un proceso hijo que finalmente mediante la llamada del sistema Exec, se ejecutará de forma totalmente independiente respecto a su proceso padre. Mediante el empleo de una tubería (buffer de memoria para la comunicación entre procesos), el proceso padre enviará un mensaje al proceso hijo:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main() {
    int pipefd[2]; // Arreglo para almacenar los descriptores de la tubería
    char mensaje[] = "Hola, proceso hijo!";

    // Crear la tubería
    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // Crear un proceso hijo
    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        exit(EXIT_FAILURE);
    }
}
```

```
if (pid == 0) {
    // Proceso hijo
    close(pipefd[1]); // Cerramos el extremo de escritura de la tubería

    char buffer[100];
    ssize_t bytes_leidos = read(pipefd[0], buffer, sizeof(buffer));

    if (bytes_leidos == -1) {
        perror("read");
        exit(EXIT_FAILURE);
    }

    buffer[bytes_leidos] = '\0'; // Agregar el carácter nulo al final

    printf("Proceso hijo recibió: %s\n", buffer);

    close(pipefd[0]); // Cerramos el extremo de lectura de la tubería en el proceso hijo
} else {
    // Proceso padre
    close(pipefd[0]); // Cerramos el extremo de lectura de la tubería

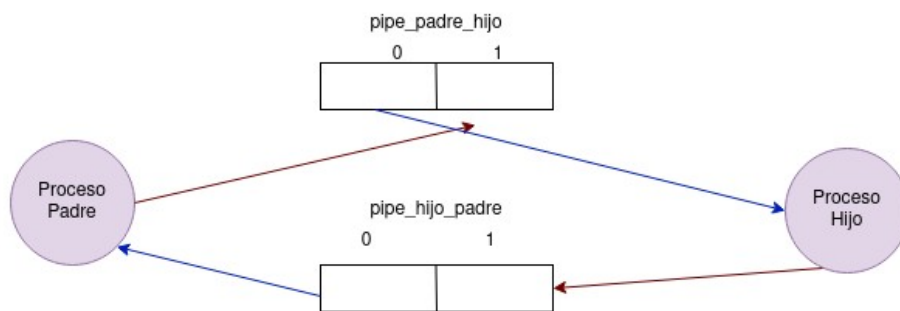
    ssize_t bytes_escritos = write(pipefd[1], mensaje, strlen(mensaje) + 1);

    if (bytes_escritos == -1) {
        perror("write");
        exit(EXIT_FAILURE);
    }

    close(pipefd[1]); // Cerramos el extremo de escritura de la tubería en el proceso padre
}
return 0;
}
```

5.2 Ejemplo 2

Programa de cálculo de suma con tubería: El proceso padre envía un número entero al proceso hijo a través de una tubería, y el proceso hijo calcula la suma de los primeros números enteros hasta ese número y envía el resultado de vuelta al proceso padre.



```
#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <stdlib.h>

int main() {

    int pipe_padre_hijo[2];

    int pipe_hijo_padre[2];

    // Crear las tuberías

    if (pipe(pipe_padre_hijo) == -1 || pipe(pipe_hijo_padre) == -1) {

        perror("Error al crear la tubería");
```

```
    exit(1);
}

pid_t pid = fork();

if (pid < 0) {
    perror("Error al hacer fork");
    exit(1);
}

// =====

// PROCESO HIJO

// =====

if (pid == 0) {
    int n, suma = 0;

    // Cerrar extremos no usados
    close(pipe_padre_hijo[1]); // No escribe
    close(pipe_hijo_padre[0]); // No lee

    // Leer el número enviado por el padre
    read(pipe_padre_hijo[0], &n, sizeof(n));

    // Calcular la suma
    for (int i = 1; i <= n; i++) {
        suma += i;
    }

    // Enviar el resultado al padre
    write(pipe_hijo_padre[1], &suma, sizeof(suma));

    // Cerrar tuberías
```

```
close(pipe_padre_hijo[0]);

close(pipe_hijo_padre[1]);

exit(0);  }

// PROCESO PADRE

else {

    int n, resultado;

    // Cerrar extremos no usados

    close(pipe_padre_hijo[0]); // No lee

    close(pipe_hijo_padre[1]); // No escribe

    printf("Ingrese un número entero: ");

    scanf("%d", &n);

    // Enviar el número al hijo

    write(pipe_padre_hijo[1], &n, sizeof(n));

    // Leer el resultado del hijo

    read(pipe_hijo_padre[0], &resultado, sizeof(resultado));

    printf("La suma de los primeros %d números es: %d\n", n, resultado);

    // Cerrar tuberías

    close(pipe_padre_hijo[1]);

    close(pipe_hijo_padre[0]);

}

return 0;

}
```

5.3 Actividades

1. Compila y ejecuta el código del ejemplo anterior.
2. **Programa de búsqueda de archivos:** El proceso padre recibe un directorio como entrada del usuario y envía el nombre del directorio al proceso hijo a través de una tubería. El proceso hijo busca archivos en ese directorio y envía la lista de archivos al proceso padre.
3. **Programa de conteo de palabras:** El proceso padre envía un fragmento de texto al proceso hijo a través de una tubería, y el proceso hijo cuenta las palabras en ese fragmento de texto y envía el resultado al proceso padre.
4. **Programa de generación de números primos:** El proceso padre envía un rango de números al proceso hijo a través de una tubería, y el proceso hijo genera y envía de vuelta la lista de números primos dentro de ese rango.
5. **Programa de comunicación bidireccional:** Tanto el proceso padre como el proceso hijo pueden enviar mensajes al otro a través de una tubería, y ambos procesos pueden recibir y mostrar los mensajes enviados por el otro proceso.
6. **Programa de transferencia de archivos:** El proceso padre envía un archivo al proceso hijo a través de una tubería, y el proceso hijo guarda el archivo en el sistema de archivos.
7. **Programa de sincronización de procesos:** El proceso padre y el proceso hijo coordinan sus acciones a través de tuberías para realizar una tarea en conjunto, como la impresión de un patrón específico en la consola.

8. **Programa de chat entre procesos:** El proceso padre y el proceso hijo actúan como dos participantes en un chat, enviándose mensajes a través de una tubería bidireccional. Ambos procesos pueden recibir y mostrar los mensajes del otro proceso en la consola.

6. Interprete de ordenes

Un interprete de ordenes es un programa que se encuentra a la espera que el usuario le indique una orden. Por ejemplo, los sistemas operativos, tienen todos sus interpretes de ordenes que permanecen atentos a las solicitudes de comandos por parte del usuario.

Veamos seguidamente un ejemplo de programa interprete de ordenes o comandos. La ejecución de los mismos será llevada a cabo por el proceso hijo:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#define MAX_INPUT_LENGTH 1024

#define MAX_ARGS 64

int main() {

    char input[MAX_INPUT_LENGTH];

    char *args[MAX_ARGS];

    int status;
```

```
while (1) {

    printf("MiShell> ");

    fgets(input, MAX_INPUT_LENGTH, stdin);

    // Eliminar el salto de línea final
    input[strlen(input) - 1] = '\0';

    // Dividir la entrada en argumentos
    char *token = strtok(input, " ");

    int i = 0;

    while (token != NULL && i < MAX_ARGS - 1) {

        args[i++] = token;

        token = strtok(NULL, " ");

    }

    args[i] = NULL; // Terminar la lista de argumentos con NULL
    if (i > 0) {
        pid_t child_pid = fork();
        if (child_pid == 0) {
            // Proceso hijo: Ejecutar el comando
            execvp(args[0], args);
            perror("execvp"); // En caso de error en execvp
            exit(EXIT_FAILURE);
        } else if (child_pid > 0) {
            // Proceso padre: Esperar a que el hijo termine
            waitpid(child_pid, &status, 0);
        } else {
            perror("fork"); // En caso de error en fork
        }
    }
    return 0;}
```

6.1 Actividades

1. Ejecuta el programa Interprete de ordenes del ejemplo anterior. Comprueba que te ofrece la posibilidad de ejecutar cualquier orden de tu sistema operativo.

7 Ejecución en segundo plano

Cuando un proceso se ejecuta en background o segundo plano se dice que es un servicio o tarea. En nuestro sistema operativo tenemos la posibilidad de visualizar qué servicios se encuentran en ejecución en segundo plano.

Un servicio o proceso en background es un proceso que se ejecuta de forma oculta para el usuario. Mientras el usuario puede seguir solicitando nuevas ordenes al sistema, los procesos en background (también conocidos como demonios) continuarán con su oculta ejecución.

A continuación vamos a ver el código de un programa en C que crea un proceso hijo. El padre esperará hasta que la ejecución del proceso hijo finalice:

```
#include <stdio.h>

#include <stdlib.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <unistd.h>

int main() {

    pid_t pid = fork();

    if (pid < 0) {
```

```
perror("fork");

exit(1);

} else if (pid == 0) {

    // Proceso hijo

    printf("Proceso hijo iniciado.\n");

    sleep(10); // Esperar 10 segundos

    printf("Proceso hijo finalizado.\n");

} else {

    // Proceso padre

    printf("Proceso padre esperando a que el proceso hijo termine...\n");

    int status;

    pid_t child_pid = wait(&status);

    if (WIFEXITED(status)) {

        printf("Proceso hijo con PID %d ha terminado con estado de salida %d.\n",
child_pid, WEXITSTATUS(status));

    } else if (WIFSIGNALED(status)) {

        printf("Proceso hijo con PID %d ha terminado debido a una señal %d.\n", child_pid,
WTERMSIG(status));

    } } return 0;}
```

7.1 Actividades

1. Compila y ejecuta en primer plano el ejemplo del programa anterior que se denominaba servicio1.c.
2. Ejecuta en segundo plano el programa ejecutable del ejemplo anterior: `./servicio1 &`
3. Comprueba que mientras se está ejecutando el proceso anterior en segundo plano, desde el interprete de ordenes de tu sistema operativo puedes seguir ejecutando otras ordenes.
4. Vuelve a ejecutar el proceso anterior en segundo plano. Mientras la ejecución de dicho proceso está teniendo efecto, prueba ejecutar el comando `ps -l` para ver los procesos activos existentes en tu sistema (entre otros podrás ver los procesos en segundo plano).

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
1	S	0	4272	4271	0	80	0	-	4334	do_pol	pts/1	00:00:00	sudo
4	S	0	4273	4272	0	80	0	-	2443	do_wai	pts/1	00:00:00	su
4	S	0	4274	4273	0	80	0	-	2097	do_wai	pts/1	00:00:00	bash
4	S	0	4594	4274	0	80	0	-	670	do_wai	pts/1	00:00:00	servicio1
1	S	0	4595	4594	0	80	0	-	670	hrtime	pts/1	00:00:00	servicio1
4	R	0	4597	4274	0	80	0	-	2882	-	pts/1	00:00:00	ps

5. Mientras se ejecuta el proceso en segundo plano anterior, contesta a las siguientes preguntas:
 - ¿Cuáles son los PIDS de los procesos padre e hijo ?
 - ¿Qué tamaño en memoria ocupan los procesos padre e hijo?
 - ¿Qué realiza la función “sleep”? ¿Qué proceso concluye antes su ejecución?
6. Modifica el código del programa para asegurar que el proceso padre imprime su mensaje de presentación (“Soy el padre...”) antes que el hijo imprima el suyo. Guarda el nuevo programa con el nombre *padrehijo2.c*.
7. Modifica el código fuente del programa declarando una variable entera llamada “varfork” e inicializándola a 10. En el padre, dicha variable deberá incrementarse 10 veces y de 10 en 10. En el hijo, “varfork” se incrementará 10 veces y de 1 en 1. Además, el programa deberá imprimir por pantalla el valor final de la variable “varfork” tanto para el padre como para el hijo. Guarda el nuevo programa con el nombre *padrehijo3.c*.